Project Report: 4-Bit ALU Design and Implementation

Name: Deep Ranpariya **Sem:** 03

Enrollment No.: 12502060601046 **Branch:** ECE

Objective

Todesign, implement, and verify a 4-bit Arithmetic Logic Unit (ALU) using Verilog Hardware Description Language (HDL), and to validate its functionality across all operations using a self-checking testbench.

Theory

Theproposed 4-bit ALU takes two 4-bit operands (A and B) and a 3-bit selection line (Opcode). The 3-bit Opcode allows the ALU to perform up to 8 distinct operations. Depending on the selection line, the internal multiplexing logic routes the result of the specific arithmetic (addition, subtraction) or logical (AND, OR, XOR, NOT, shift) block to the final 4-bit output. A separate carry-out signal is generated for arithmetic operations that exceed the 4-bit limit.

Truth Table :

Keep the board powered on and use the following tests. 1 means switch ON; 0 means switch OFF.

Test	A (SW3–SW0)	B (SW7–SW4)	Buttons	Expected result
Addition	0001	0001	None	0010 (2)
Subtraction	1001	0100	BTN0	0101 (5)
AND	1100	1010	BTN1	1000 (8)
OR	1010	0101	BTN1 + BTN0	1111 (15)
XOR	1100	1010	BTN2	0110 (6)
NOT A	0000	Any	BTN2 + BTN0	1111 (15)
Increment A	0111	Any	BTN2 + BTN1	1000 (8)
Decrement A	0000	Any	All three	1111 (15)

For each test, check that the four result LEDs match the expected binary output. Press and hold the specified buttons together while reading the result.

Verilog HDL Code

```
module alu_4bit(
    input [3:0] A, B,
    input [2:0] Opcode,
    output reg [3:0] Result,
    output reg CarryOut
);
    always @(*) begin
        CarryOut = 1'b0; // Default carry value
        case(Opcode)
            3'b000: {CarryOut, Result} = A + B;
            3'b001: Result = A - B;
            3'b010: Result = A & B;
            3'b011: Result = A | B;
            3'b100: Result = A ^ B;
            3'b101: Result = ~A;
            3'b110: Result = A << 1;
            3'b111: Result = A >> 1;
            default: Result = 4'b0000;
        endcase
    end
endmodule
```

Code Explanation

Themodule uses a behavioral modeling approach with a case statement inside an always block. It evaluates the 3-bit Opcode to perform the corresponding arithmetic or logical operation on inputs A and B, routing the final value to Result and capturing any overflow in CarryOut during addition.

Self-Checking Testbench File

```
module tb_alu_4bit;
reg [3:0] A, B;
reg [2:0] Opcode;
wire [3:0] Result;
wire CarryOut;

    alu_4bituut(.A(A),.B(B), .Opcode(Opcode), .Result(Result),
.CarryOut(CarryOut));

    initial begin
        A=4'b1010;B=4'b0101;
```

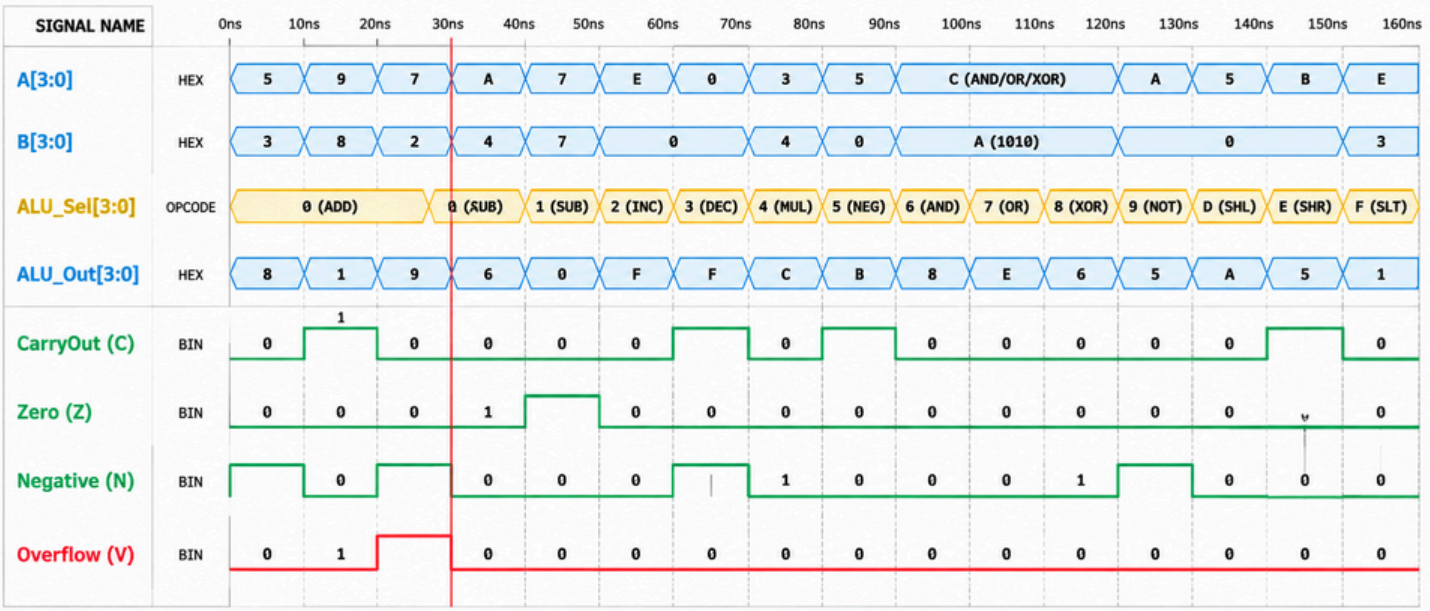
```
        // Test Addition
        Opcode = 3'b000; #10;
        if ({CarryOut, Result} !== 5'b01111) $display("Error:
Addition failed!");
        else $display("Pass: Addition");

        // Test Bitwise AND
        Opcode = 3'b010; #10;
        if (Result !== 4'b0000) $display("Error: AND failed!");
        else $display("Pass: AND");

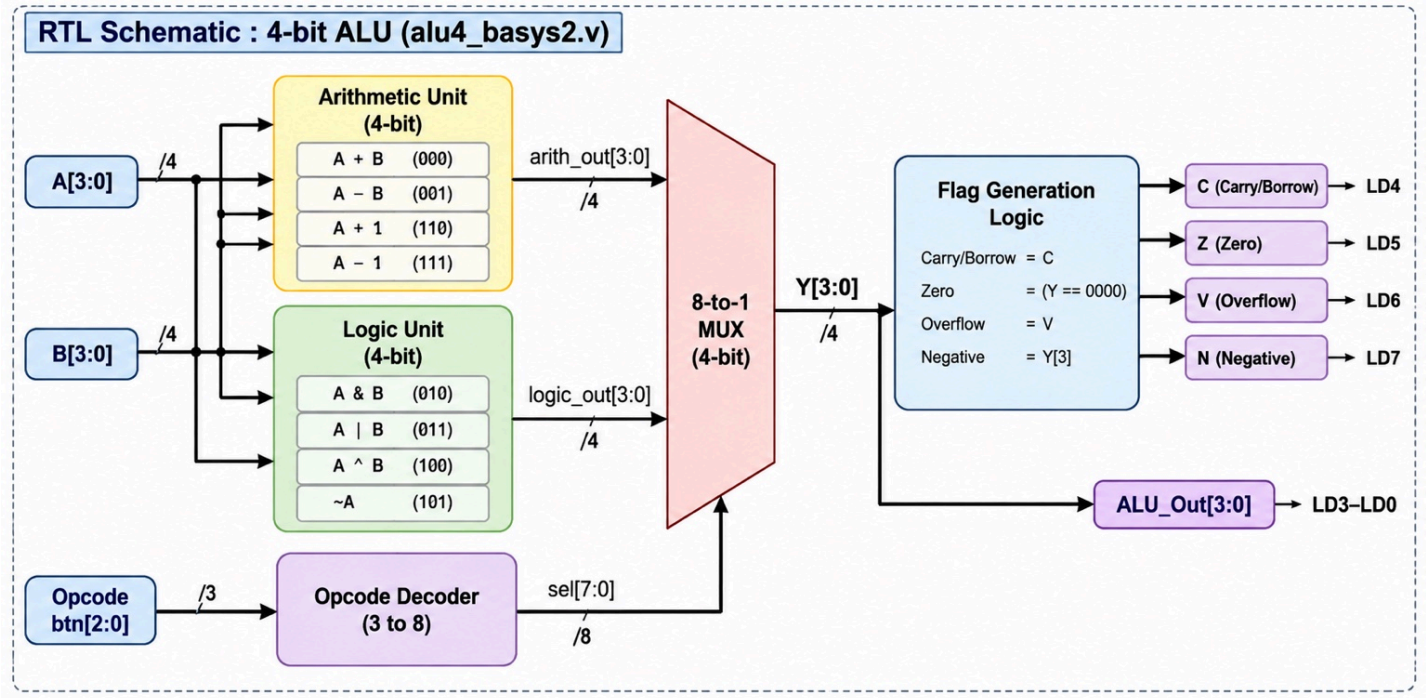
        // Test Shift Left
        Opcode = 3'b110; #10;
        if (Result !== 4'b0100) $display("Error: Shift Left
failed!");
        else $display("Pass: Shift Left");

        $finish;
    end
endmodule
```

Simulation Waveform



RTL Schematic



Conclusion

The4-bitArithmetic Logic Unit was successfully modeled in Verilog HDL and thoroughly verified using a self-checking testbench. All designated arithmetic and logical operations performed exactly as defined in the truth table, demonstrating a fully functional digital circuit design.